

Title

Report on Assignment 3

Aaron Bussche

Jason Kasdiran

David Magaram

Group 28

1 Part A

1.1 Reproducing quantization

1.2 Finding optimal color palette

1.3 Results

2 Part B

2.1 Introduction

Follow the README to see how to get images and quantitative data.

Alignment – Each boid tries to match the vector (speed and direction) of the other boids around it. Coherence controls how quickly they try to match vectors.

Coherence – Each boid tries to move towards the center of mass of the other boids around it. Coherence controls how quickly they try to move towards the center of mass.

Separation – Each boid tries to keep a certain distance from the other boids around it to not run into them. If boids get too close, they will steer away. Separation controls how much they try to keep this distance.

Visual range – The distance within which a boid can see other boids and be influenced by them. If the visual range is too small, the boids will not be able to see enough other boids to form a cohesive flock. If it is too large, they may be influenced by too many boids and not form a cohesive flock.

Explain how the implementation satisfies all the requirements - fixed speed,

The model keeps Boid speeds fixed; agents should update their direction but move at a constant speed v which is a parameter of your model. (`const move = this.multiplyVector(this.dir, this.speed)`)

Explicitly considers what should happen at the boundary of your simulation field: The boundary is wrapped. When a boid exits from one side, it reappears on the opposite side.

uses two interaction radii: one for separation, and one for cohesion/alignment. (Ask yourself: which one should be smaller and why?): The inner radius is used for separation, while the outer radius is used for cohesion and alignment. The inner radius should be smaller than the outer radius because separation requires a closer distance to trigger the behavior, while cohesion and alignment can occur at a larger distance.

excludes the boid itself from its own neighborhood when calculating the update rules, so it does not try to separate, align or cohere to itself: The implementation excludes the boid itself from its own neighborhood when calculating the update rules. This is done by ensuring that when iterating through the neighboring boids, the current boid is not included in the calculations for separation, alignment, and cohesion. (`if( p.id == x.id ) continue`)

2.2 Methods

2.3 Results

It is possible to reach high alignment in 300 steps with these parameters.

Parameter	Value
$Radius_{\text{inner}}$	10
$Radius_{\text{outer}}$	25
Alignment weight	1
Coherence weight	1, 0.5
Separation weight	1
Number of boids	200

Table 1: Parameters.

Time	0	100	200	300
Order parameter	0.0208	0.0753	0.105	0.616
Nearest-neighbor distance	13.956	5.925	4.278	3.405

Table 2: Quantitative results for default parameters.

Time	0	100	200	300
Order parameter	0.096	0.509	0.751	0.695
Nearest-neighbor distance	14.466	9.076	8.053	7.506

Table 3: Quantitative results for alternate parameters.

The visualizations in Figure 1 show the positions of the boids at different time steps. Four snapshots were taken at time steps 0, 100, 200, and 300, and are separated into an initial state, final state, and two intermediate states.

Red color represents the inner radius, while the black bar coming out of it indicates in which direction each boid is facing. The outer radius is shown as a gray circle and is each boids' interaction range.

200 boids were initialized randomly at a 400x400 field with wrapped boundaries. Boids then move around, updating their positions and orientations based on the alignment, coherence, and separation rules. Boids interact with each other by coming in each other interaction range, forming flocks as they align and cohere with each other, while also maintaining a certain distance to avoid collisions. Over time, larger flocks form as boids and smaller flocks join together. However, as shown in Figure 1d, not all boids are able to find a flock by time step 300 and remain isolated until they bump into another boid or flock.

The boids are distributed across the simulation field, and their positions and orientations change over time as they interact with each other based on the alignment, coherence, and separation rules.

Over time, we can observe the formation of flocks as the boids align and cohere with each other. The separation behavior can also be seen as boids maintain a certain distance from each other to avoid collisions.

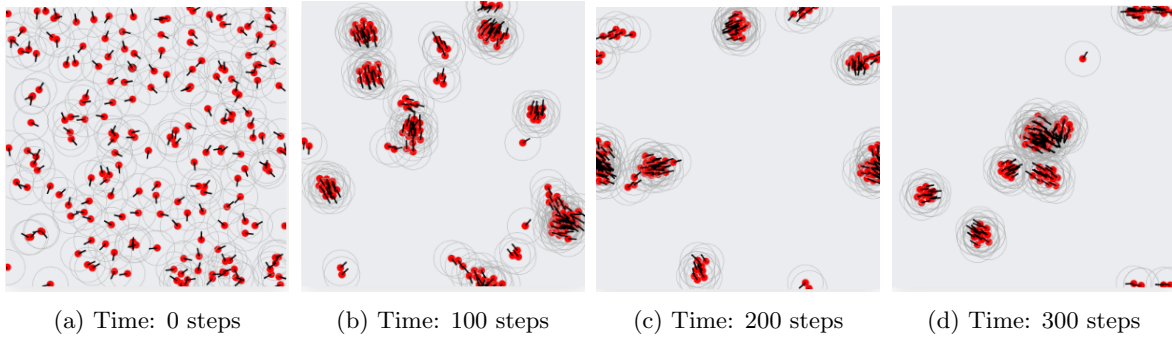


Figure 1: Visualizations of boids at four different time steps.

2.4 Experimental design

The Table 3 below displays all the parameter configurations that were explored in order to investigate the effect of different parameters on the behavior of the system. These parameters were chosen because they allow for the exploration of all interesting configurations of the system.

By exploring different parameter configurations, interesting behaviors can be observed. By setting the coherence weight to 0, boids no longer strive to move towards the center of mass of their neighbors, instead, they align with their neighbors and keep a certain distance from them. This leads to an interesting behavior where boids form large nets that span the entire simulation field. Because of that, after a short while all boids start moving in the same direction. By increasing the outer radius, this can be sped up as boids are initialized with larger interaction radius and start forming large nets sooner. On the other hand, if the inner radius is increased to be more than half the value of the outer radius, this uniform behavior is disturbed. Ultimately, coherence weight plays a role in the sizes of the flocks that are formed, where higher values lead to larger flocks and lower values lead to tighter flocks.

Parameter	Value
$Radius_{\text{inner}}$	10, 40, 70, 100
$Radius_{\text{outer}}$	25, 50, 75, 100
Alignment weight	0, 0.5, 1
Coherence weight	0, 0.5, 1
Separation weight	0, 0.5, 1
Number of boids	100, 200, 300

Table 4: Parameters.